TÀI LIỆU MÔ TẢ THIẾT KẾ PHẦN MỀM

(Software Design Descriptions - SDD)

Hệ thống Secure Gateway mã hóa dữ liệu IP thời gian thực sử dụng STM32

Phiên bản 1.0.1

Được soạn bởi: Vũ Văn Tiến

Hà Nội, tháng 4, năm 2026

MỤC LỤC

1. GIỚI THIỆU 1
1.1. Mục đích 1
1.2. Phạm vi 1
1.3. Đối tượng sử dụng 2
1.4. Tóm tắt 2
1.5. Mã nguồn dự án 3
2. TỔNG QUAN HỆ THỐNG 4
2.1. Kiến trúc phân tầng 4
2.2. Luồng dữ liệu tổng thể 4
2.3. Nguyên tắc thiết kế cốt lõi 5
2.3.1. Cơ chế Zero-copy 5
2.3.2. Cấp phát bộ nhớ tĩnh 5
2.3.3. Xử lý dựa trên sự kiện 5
3. THIẾT KẾ CHI TIẾT CÁC MODULE 5
3.1. Module Driver ENC28J60 5
3.1.1. Cấu trúc dữ liệu và định nghĩa 5
3.1.2. Đặc tả các hàm API chính 6
3.1.3. Cơ chế đồng bộ và an toàn 8
3.2. Module quản lý bộ đệm 8
3.2.1. Cấu trúc dữ liệu và định nghĩa 8
3.2.2. Thiết kế an toàn đa nhiệm 11
3.3. Module phân tích và định tuyến gói tin 11
3.3.1. Các hằng số và ràng buộc 11
3.3.2. Đặc tả luồng phân tích gói tin 12
3.3.3. Đặc tả hàm cập nhật Header 13
3.4. Module mã hóa AES 14
3.4.1. Đặc tả thuật toán và cấu trúc dữ liệu 14
3.4.2. Tối ưu hóa bộ nhớ phần cứng 15

3.4.3. Đặc tả các hàm API chính 15
3.4.4. Trình tự thực thi mã hóa 17
3.5. Thiết kế đa nhiệm 17
3.5.1. Danh sách các Task 18
3.5.2. Giao diện các Task 19
3.5.3. Chi tiết tương tác giữa các Task 19
3.6. Thiết kế Queue 20
3.6.1. Khởi tạo và cấu trúc bộ nhớ 20
3.6.2. Đặc tả Queue nhận 21
3.6.3. Đặc tả Queue gửi 22
3.6.4. Các nguyên tắc thiết kế cốt lõi 22
4. THIẾT KẾ ĐA NHIỆM VÀ GIẢI QUYẾT TƯƠNG TRANH 23
4.1. Cơ chế đồng bộ hóa 23
4.1.1. Đồng bộ ngắt ngoài 23
4.1.2. Đồng bộ truyền tải bộ nhớ 23
4.2. Cơ chế bảo vệ tài nguyên 23
4.2.1. Bảo vệ Bus giao tiếp 24
4.2.2. Bảo vệ bộ đệm tĩnh 24
4.3. Cơ chế chống bế tắc 24
4.3.1. Timeout cho Mutex/Semaphore 24
4.3.2. Loại trừ vòng lặp lồng 25
4.4. Cơ chế giám sát đa nhiệm 25
4.4.1. Cơ chế lập lịch 25
4.4.2. Lập lịch ưu tiên độc chiếm 26
4.4.3. Lập lịch đồng mức ưu tiên 26
4.4.4. Quản lý trạng thái và tối ưu hóa CPU 26
5. BIỂU ĐỒ TUẦN TỰ 27
5.1. Mô tả tương tác hệ thống 27
5.2. Biểu đồ UML 28

6. THIẾT KẾ VÀ PHÂN BỔ BỘ NHỚ 29
6.1. Phân bổ bộ nhớ RAM 29
6.2. Phân bổ bộ nhớ chương trình 29
7. CƠ CHẾ XỬ LÝ LỖI VÀ NGOẠI LỆ 30
7.1. Các điều kiện hủy gói tin 30
7.1.1. Cạn kiệt bộ đệm tĩnh 30
7.1.2. Quá tải Queue 31
7.1.3. Gói tin không hợp lệ 31
7.2. Chiến lược phục hồi 31
7.2.1. Phục hồi ngay lập tức 32
7.2.2. Cơ chế không thử lại 32
8. MA TRẬN TRUY VẾT YÊU CẦU 32
9. HẠN CHẾ CỦA HỆ THỐNG VÀ ĐÁNH ĐỔI THIẾT KẾ 35
9.1. Giới hạn giao thức hỗ trợ 35
9.2. Rủi ro từ Vector khởi tạo cố định 36

MỤC LỤC HÌNH ẢNH

Hình 3.1.1.1: Cấu trúc dữ liệu ENC28J60_Config 6
Hình 3.2.1.1: Cấu trúc dữ liệu PacketBuffer và các định nghĩa 9
Hình 3.2.1.2: Các biến tĩnh nội bộ 9
Hình 3.4.1.1: Cấu trúc dữ liệu AES_ctx 14
Hình 3.5.2.1: Nguyên mẫu hàm của các Task 19
Hình 3.6.1.1: Danh sách các Queue và khởi tạo 21
Hình 5.2.1: Biểu đồ UML quá trình xử lý một gói tin 28

MỤC LỤC BẢNG

Bảng 3.5.1.1: Danh sách các Task 19
Bảng 6.1.1: Phân bổ dung lượng RAM 29
Bảng 6.2.1: Phân bổ dung lượng Flash 30
Bảng 8.1: Ma trận truy vết các yêu cầu chức năng (FR) 33
Bảng 8.2: Ma trận truy vết các yêu cầu phi chức năng (NFR) 34
Bảng 8.3: Ma trận truy vết quyết định kiến trúc (ADR) 35

1. GIỚI THIỆU

1.1. Mục đích

Tài liệu này mô tả thiết kế chi tiết (low-level design) của hệ thống Secure Gateway AES-STM32. Trọng tâm của tài liệu này bao gồm việc thiết kế cấu trúc các module, thiết kế dữ liệu, thiết kế các hàm, cơ chế xử lý song song và luồng xử lý gói tin.

Mục tiêu cốt lõi là chuyển đổi các chức năng (FR) từ tài liệu SRS thành một bản thiết kế thực thi được. Tài liệu này đóng vai trò hướng dẫn trực tiếp cho đội ngũ phát triển Firmware, kiểm thử hệ thống và phát triển hệ thống trong tương lai. Đây là căn cứ để xây dựng kịch bản kiểm thử hệ thống (Test Cases).

1.2. Phạm vi

Thiết kế tập trung vào việc triển khai trên vi điều khiển STM32F103C8T6 (Cortex-M3) để thực hiện chức năng bảo mật (Secure Gateway) ở lớp 2/3. Phạm vi thiết kế bao gồm:

- Giao tiếp SPI với hai module ENC28J60, quản lý ngắt và truyền dữ liệu sử dụng DMA.
- Phân tích và chỉnh sửa các thành phần như Ethernet Frame, IPv4 Header, UDP Header và Payload.
- Thực hiện thuật toán mã hóa/giải mã AES-128 theo chuẩn CBC (Cipher Block Chaining) với cơ chế đệm PKCS#7.
- Quản lý đa nhiệm thông qua FreeRTOS, điều phối luồng dữ liệu bằng cơ chế Queue và quản lý bộ nhớ bằng Static Buffer Pool.

Lưu ý: Thiết kế này không bao gồm việc triển khai cho giao thức TCP/IP và các giao thức định tuyến động phức tạp khác. Chỉ dùng để truyền nhận các gói tin UDP/IP, ARP và ICMP.

1.3. Đối tượng sử dụng

Tài liệu này được biên soạn dành cho:

- Nhóm phát triển Firmware sử dụng để làm tài liệu hướng dẫn lập trình các module chi tiết.
- Nhóm kiểm thử sử dụng để kiểm tra tính đúng đắn của logic thiết kế so với yêu cầu của hệ thống.
- Nhóm bảo trì sử dụng để hiểu rõ cấu trúc bộ nhớ và luồng dữ liệu để thực hiện sửa lỗi khi có vấn đề hoặc nâng cấp trong tương lai.

1.4. Tóm tắt

[1] IEEE Std 1016™-2009, “IEEE Standard for Information Technology—Systems Design— Software Design Descriptions,” 20 July 2009.

[2] STMicroelectronics, "STM32F103x8 STM32F103xB Datasheet," Rev. 20, 2025.

[3] STMicroelectronics, "RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM®-based 32-bit MCUs," Rev. 21, 2021.

[4] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," FIPS PUB 197-upd1, May 9, 2023.

[5] Microchip Technology Inc., "ENC28J60 Data Sheet: Stand-Alone Ethernet Controller with SPI Interface," 2012.

[6] Richard Barry and The FreeRTOS Team, "Mastering the FreeRTOS™ Real Time Kernel: A Hands-On Tutorial Guide," Release Version 1.1.0.

[7] Information Sciences Institute, "Internet Protocol (RFC 791)," University of Southern California, September 1981.

[8] Internet Engineering Task Force (IETF), “User Datagram Protocol (RFC 768),” J. Postel, 28 August 1980.

[9] “An Ethernet Address Resolution Protocol,” David C. Plummer, November 1982.

[10] Tiny-AES-C Library, https://github.com/kokke/tiny-AES-c.git

[11] Microchip Technology Inc., “ENC28J60 Silicon Errata and Data Sheet Clarification,” 2010.

1.5. Mã nguồn dự án

Toàn bộ mã nguồn của dự án Secure Gateway, bao gồm cả Firmware cho STM32, Driver ENC28J60, thư viện AES (bản tối giản) và script mô phỏng trên Python, được quản lý công khai trên github.

- Link Repository: https://github.com/TIENVUZ-tech/Secure-Gateway-AES-STM32.git.
- Nhánh chính (Branch): main (nhánh duy nhất)
- Hội đồng đánh giá có thể truy cập mã nguồn để xem chi tiết cách thức triển khai các module được mô tả trong tài liệu này.

2. TỔNG QUAN HỆ THỐNG

2.1. Kiến trúc phân tầng

Hệ thống được thiết kế theo mô hình phân tầng với các chức năng nằm tách biệt giữa các tầng giao tiếp phần cứng và tầng xử lý logic. Điều này giúp tăng khả năng bảo trì và kiểm thử độc lập giữa các module.

- Tầng Driver: Chịu trách nhiệm cấu hình ngoại vi SPI, quản lý các chân ngắt (EXTI) và tương tác trực tiếp với các thanh ghi của ENC28J60.
- Tầng quản lý dữ liệu: Cung cấp cơ chế điều phối gói tin giữa các Task thông qua bộ đệm tĩnh (Static Pool) để đảm bảo tính an toàn bộ nhớ và tránh bị phân mảnh.
- Tầng xử lý giao thức: Thực hiện bóc tách Header IP/UDP và kiểm tra tính toàn vẹn của gói tin.
- Tầng ứng dụng và bảo mật: Thực thi mã hóa AES-128-CBC và ra quyết định chuyển tiếp gói tin.

2.2. Luồng dữ liệu tổng thể

Luồng dữ liệu trong Secure Gateway được chia thành các bước sau:

Bước 1 – Nhận gói: Module mạng nhận tín hiệu vật lý khi có gói tin đến → Kích hoạt ngắt → RX Task đọc dữ liệu qua SPI vào Buffer của vi điều khiển.

Bước 2 – Phân loại: Kiểm tra cấu trúc gói tin thông qua phần Header. Nếu gói tin không phải UDP hoặc không phải những gói tin không cần mã hóa như ARP hoặc ICMP thì gói tin sẽ bị hủy bỏ ngay.

Bước 3 – Thực thi bảo mật: Payload của gói tin được mã hóa bằng AES-128-CBC. Header IP/UDP được cập nhật (độ dài mới, Checksum mới).

Bước 4 – Truyền gói: Gói tin đã mã hóa được đẩy vào Queue TX Task để gửi ra cổng mạng còn lại.

2.3. Nguyên tắc thiết kế cốt lõi

Để đáp ứng các yêu cầu khắt khe về thời gian thực và độ ổn định trên dòng vi điều khiển có tài nguyên hạn chế như STM32F103C8T6, thiết kế tuân thủ 3 nguyên tắc sau:

2.3.1. Cơ chế Zero-copy

Hệ thống không thực hiện sao chép dữ liệu gói tin giữa các tầng xử lý. Thay vào đó, chỉ có con trỏ (Pointer) trỏ tới địa chỉ vùng đệm Static Buffer Pool được truyền qua hai Queue (FreeRTOS Queues). Điều này giúp tiết kiệm tài nguyên CPU và tối ưu hóa tốc độ xử lý.

2.3.2. Cấp phát bộ nhớ tĩnh

Toàn bộ vùng nhớ dành cho gói tin và ngăn xếp (Stack) của các Task được xác định ngay tại thời điểm biên dịch. Thiết kế này loại bỏ hoàn toàn sử dụng các hàm cấp phát/thu hồi bộ nhớ như malloc()/free(), từ đó triệt tiêu rủi ro tràn bộ nhớ (Stack Overflow) hoặc phân mảnh bộ nhớ (Memory Fragmentation) trong quá trình vận hành lâu dài.

2.3.3. Xử lý dựa trên sự kiện

Thay vì sử dụng cơ chế thăm dò (Polling) gây lãng phí tài nguyên (do CPU phải liên tục kiểm tra các trạng thái trên các ngoại vi), hệ thống hoạt động hoàn toàn dựa trên tín hiệu ngắt từ phần cứng ENC28J60. Các Task sẽ ở trạng thái Blocked cho đến khi có dữ liệu thực tế, giúp CPU có thời gian xử lý các Task nền hoặc tiết kiệm năng lượng.

3. THIẾT KẾ CHI TIẾT CÁC MODULE

3.1. Module Driver ENC28J60

3.1.1. Cấu trúc dữ liệu và định nghĩa

Hệ thống sử dụng cấu trúc ENC28J60_Config để quản lý trạng thái độc lập của từng module mạng (vì hệ thống dùng 2 module song song).

```c
// Configure structure
typedef struct {
    SPI_HandleTypeDef *hspi; // SPI peripheral handle (SPI1 or SPI2)
    GPIO_TypeDef *NSS_Port; // Negative slave Select Port
    uint16_t NSS_Pin; // Negative slave Select Pin
    uint16_t next_packet_ptr; // Pointer managing the position of the next packet
    GPIO_TypeDef *RST_Port; // Reset Port
    uint16_t RST_Pin; // Reset Pin
    uint8_t current_bank; // Store the current register bank
} ENC28J60_Config;
```

Hình 3.1.1.1: Cấu trúc dữ liệu ENC28J60_Config

Phân bổ bộ nhớ đệm nội bộ:

- RX_START (0x0000) đến RX_END (0x19FF): Dành 6.5KB cho bộ đệm nhận gói tin.
- TX_START (0x1A00) đến hết: Vị trí bắt đầu bộ đệm truyền gói tin.

3.1.2. Đặc tả các hàm API chính

Dựa trên triển khai thực tế, các hàm giao tiếp chính bao gồm:

a. Hàm khởi tạo hệ thống

Tên hàm: ENC28J60_Init

Nguyên mẫu: void ENC28J60_Init (ENC28J60_Config *spi, uint8_t *mac_address);

Logic xử lý:

- Thực hiện Reset cứng qua chân RST và Reset mềm qua lệnh ENC28J60_SYS_RST_CMD.
- Cấu hình vùng nhớ Ring Buffer (ERXST, ERXND, ERXRDPT).
- Thiết lập bộ lọc MAC (ERXFCON_CRCEN) và độ dài gói tin tối đa (MAMXFL = 598 bytes)
- Cấu hình lớp vật lý (PHY) ở chế độ Full Duplex và đèn LED trạng thái (gồm LED hiển thị trạng thái kết nối và LED báo hiệu có hoạt động truyền/nhận).

- Kích hoạt ngắt nhận gói tin (EIE_PKTIE).

b. Hàm nhận gói tin

Tên hàm: ENC28J60_ReceivePacket

Nguyên mẫu: uint16_t ENC28J60_ReceivePacket (ENC28J60_Config *spi, uint8_t *pBuffer, uint16_t max_length);

Logic thiết kế:

- Sử dụng Mutex để chiếm quyền sử dụng bộ SPI tương ứng.
- Kiểm tra số lượng gói tin hiện có qua thanh ghi EPKTCNT.
- Đọc 6 bytes Header (bao gồm next_ptr, len, rxstat) để xác định tính hợp lệ của gói tin.
- Sử dụng HAL_SPI_Receive_DMA để đọc Payload vào RAM.
- Sử dụng xSem_DMA_SPI1/2_Done với Timeout 50ms để đồng bộ hóa quá trình DMA.
- Cập nhật ERXRDPT (phải là số lẻ) và giảm EPKTCNT qua việc đặt bit ECON2_PKTDEC lên mức 1.
- Giải phóng Mutex sau khi việc nhận gói tin kết thúc.

c. Hàm gửi gói tin

Tên hàm: ENC28J60_SendPacket

Nguyên mẫu: void ENC28J60_SendPacket (ENC28J60_Config *spi, uint8_t *packet_data, uint16_t length);

Logic thiết kế:

- Sử dụng Mutex để chiếm quyền sử dụng bộ SPI tương ứng và giải phóng khi dùng xong.
- Xử lý Errata #12 bằng cách kiểm tra trạng thái cờ ECON1_TXRTS. Nếu cờ này vẫn bật (đang truyền) quá 10ms hoặc nếu phát hiện lỗi truyền (EIR_TXERIF), thực hiện Reset cục bộ module truyền bằng cách set và clear cờ ECON1_TXRST.

- Ghi địa chỉ bắt đầu vùng nhớ truyền (TX_START = 0x1A00) vào thanh ghi EWRPT. Ghi địa chỉ kết thúc (TX_START + length) vào ETXND.
- Truyền dữ liệu Header và Payload sử dụng HAL_SPI_Transmit_DMA để đưa dữ liệu từ RAM của STM32 vào Buffer của ENC28J60.
- Sử dụng xSem_DMA_SPI1/2_Done với Timeout 50ms để đồng bộ hóa quá trình DMA. Thiết lập cờ ECON1_TXRTS để chip bắt đầu phát tín hiệu.
- Giải phóng Mutex sau khi kết thúc việc truyền gói tin.

3.1.3. Cơ chế đồng bộ và an toàn

Mọi truy cập vào Bus SPI thông qua Driver đều phải chiếm giữ spi1_mutex hoặc spi2_mutex với thời gian chờ (Timeout) là 200ms để tránh tranh chấp giữa các Task RX, TX và Heartbeat.

Hàm ENC28J60_ClearErrors được gọi định kỳ để xóa cờ EIR_RXERIF, để cổng nhận không bị đóng băng khi có hiện tượng ngập lụt gói tin (Flood).

3.2. Module quản lý bộ đệm

Module này đóng vai trò cốt lõi trong việc quản lý bộ nhớ của hệ thống, cung cấp cơ chế cấp phát bộ đệm tĩnh cho gói tin mạng nhằm triệt tiêu hiện tượng phân mảnh bộ nhớ (Memory Fragmentation).

3.2.1. Cấu trúc dữ liệu và định nghĩa

Thiết kế sử dụng cơ chế tách biệt giữa dữ liệu bộ đệm và trạng thái để tối ưu hóa việc kiểm soát.

```c
#define BUFFER_COUNT 10
#define BUFFER_SIZE 614

extern osMutexId pool_mutex;

typedef enum {
    BUFFER_FREE = 0,
    BUFFER_IN_USE = 1
} BufferStatus;

typedef struct {
    uint8_t data[BUFFER_SIZE]; // Static memory
    uint8_t source_spi; // 1 = SPI1, 2 = SPI2
    uint16_t length; // Length of the data
} PacketBuffer;
```

Hình 3.2.1.1: Cấu trúc dữ liệu PacketBuffer và các định nghĩa

```c

// Static memory
static PacketBuffer pool_buffers[BUFFER_COUNT];
static BufferStatus pool_status[BUFFER_COUNT];

```

Hình 3.2.1.2: Các biến tĩnh nội bộ

Đặc tả các hàm API chính

a. Hàm khởi tạo bộ đệm

Tên hàm: BufferPool_Init

Nguyên mẫu: void BufferPool_Init(void);

Logic thiết kế:

- Duyệt qua toàn bộ danh sách pool_status và thiết lập tất cả các Buffer về trạng thái BUFFER_FREE.
- Reset toàn bộ các trường dữ liệu điều khiển (length, source_spi) của từng bộ đệm về giá trị 0.
- Đảm bảo hệ thống bắt đầu với trạng thái bộ nhớ sạch trước khi kích hoạt các Task trong FreeRTOS.

b. Hàm cấp phát bộ đệm

Tên hàm: BufferPool_Acquire

Nguyên mẫu: PacketBuffer* BufferPool_Acquire(void);

Logic thiết kế:

- Thực hiện chiếm giữ pool_mutex với thời gian chờ (Timeout) 10ms để đảm bảo tính độc quyền khi truy cập vào bảng trạng thái.
- Duyệt mảng pool_status để tìm phần tử đầu tiên có trạng thái BUFFER_FREE.
- Nếu tìm thấy BUFFER_FREE thì đánh dấu là BUFFER_IN_USE, reset length và source_spi, sau đó trả về địa chỉ của bộ đệm đó.
- Nếu không tìm thấy (Pool đã đầy) thì trả về NULL.
- Giải phóng pool_mutex trước khi kết thúc hàm.

Độ phức tạp: Thời gian O(n), không gian O(1).

c. Hàm giải phóng bộ đệm

Tên hàm: BufferPool_Release

Nguyên mẫu: void BufferPool_Release(PacketBuffer *buffer);

Logic thiết kế:

- Kiểm tra con trỏ đầu vào, nếu NULL thì kết thúc hàm ngay lập tức để tránh lỗi Runtime.
- Chiếm giữ pool_mutex với cơ chế chờ vô hạn (osWaitForever) để đảm bảo bộ đệm chắc chắn được thu hồi.

- Xác định chỉ số (index) của buffer trong mảng pool_buffers thông qua duyệt mảng và so sánh địa chỉ.
- Cập nhật trạng thái tương ứng trong pool_status về BUFFER_FREE.
- Giải phóng pool_mutex.

Độ phức tạp: Thời gian O(n), không gian O(1).

3.2.2. Thiết kế an toàn đa nhiệm

Do BufferPool_Release và BufferPool_Acquire được gọi từ nhiều Task khác nhau (RX Task, Processing Task, TX Task), việc sử dụng pool_mutex để bảo vệ dữ liệu khi truy cập là bắt buộc để tránh tình trạng Race Condition (nhiều Task cùng chiếm dụng một bộ đệm).

3.3. Module phân tích và định tuyến gói tin

Module này hoạt động như một bộ lọc và định tuyến cốt lõi của Gateway. Nó chịu trách nhiệm bóc tách các lớp mạng (Lớp 2 – Data Link Layer, Lớp 3 – Network Layer, Lớp 4 – Transport Layer), kiểm tra tính hợp lệ của gói tin và quyết định luồng di chuyển của gói tin (chuyển tiếp, đưa vào module mã hóa, hủy bỏ). Toàn bộ logic được thực thi bên trong vPacket_Processing_Task.

3.3.1. Các hằng số và ràng buộc

Do hạn chế về tài nguyên phần cứng của vi điều khiển STM32F103C8T6 (chỉ 20KB RAM) nên hệ thống không thể xử lý quá nhiều gói tin và kích thước của mỗi gói tin không được quá lớn.

- PAYLOAD_MAX_SIZE = 512: Kích thước tối đa của phần dữ liệu hữu ích (Payload) được phép xử lý.
- Chỉ chấp nhận các gói tin IPv4 không bị phân mảnh. (Unfragmented packets).

3.3.2. Đặc tả luồng phân tích gói tin

Quá trình phân tích và thực hiện truy xuất thông tin được thực hiện trực tiếp trên vùng đệm PacketBuffer thông qua các chỉ số (offset) cố định trong giao thức mạng, đảm bảo độ phức tạp O(1) cho mỗi lần kiểm tra.

a. Lọc Lớp 2 (Data Link Layer)

Thao tác: Đọc trường EtherType tại offset 12-13.

Xử lý:

- Nếu là 0x0806 (ARP): Gói tin được chuyển tiếp trực tiếp vào xTX_Queue (không cần mã hóa) để hai thiết bị đầu cuối có thể tìm thấy địa chỉ MAC của nhau.
- Nếu là 0x0800 (IPv4): Đạt điều kiện, chuyển sang bước bóc tách Lớp 3.
- Gói tin thuộc các chuẩn khác: Gọi BufferPool_Release để hủy gói.

b. Lọc Lớp 3 và 4 (Network & Transport Layer)

Chuẩn hóa độ dài: Đọc Total Length của IPv4 (offset 16-17). Tính toán kích thước thực sự của gói tin (không lấy trực tiếp chiều dài của khối dữ liệu nhận được vì trong đó còn chứa các byte đệm do cơ chế Auto-padding của Ethernet tự động chèn vào).

$$packet \rightarrow length = 14 + real_ip_length$$

(*Trong đó 14 là kích thước của phần Ethernet Header*)

Phân loại giao thức (offset 23):

- Nếu là 0x01 (ICMP): Gói tin được chuyển ngay vào xTX_Queue để duy trì khả năng kiểm tra kết nối (Ping).
- Nếu là 0x11 (UDP): Tiến hành kiểm tra cờ phân mảnh (Fragment Offset) tại offset 20-21. Nếu bit phân mảnh được bật (flag_offset & 0x3FFF) thì tiến hành hủy gói tin.

c. Kiểm tra ranh giới dữ liệu

Tính toán chiều dài IP Header (IHL = (offset 14 & 0x0F) *4).

Xác định vị trí bắt đầu của Payload UDP (udp_payload_offset = 14 + IHL + 8).

(Trong đó 8 là kích thước của UDP Header)

Nếu chiều dài phần Payload vượt quá PAYLOAD_MAX_SIZE (512 bytes) hoặc cấu trúc gói tin bị lỗi hụt byte (kích thước của gói tin <= udp_payload_offset (kích thước của gói tin nhỏ hơn phần Header tiêu chuẩn)), hệ thống lập tức tiến hành hủy gói tin.

3.3.3. Đặc tả hàm cập nhật Header

Sau khi gói tin trải qua module Mã hóa (AES) và bị thay đổi kích thước do cơ chế đệm (Padding), cấu trúc Header phải được cập nhật trước khi đưa ra mạng. Công việc này được thực hiện bởi hàm Update_Ip_Checksum.

Nguyên mẫu: void Update_Ip_Checksum(PacketBuffer *packet, uint8_t ip_header_length);

Logic thiết kế:

- Cập nhật chiều dài mới: Ghi đè UDP Length mới vào offset 14 + IHL + 4 và IPv4 Total Length vào offset 16-17.
- Tính toán lại IPv4 Checksum: Xóa trường Checksum cũ (offset 24-25) về 0x0000. Sử dụng vòng lặp để tính tổng bù 1 (One's complement sum) của toàn bộ các từ 16-bit trong IPv4 Header. Xử lý bit nhớ (carry) bằng phép toán sum = (sum & 0xFFFF) + (sum >> 16). Ghi giá trị nghịch đảo bit (~sum) vào lại offset 24-25.
- Bỏ qua UDP Checksum: Nhằm tối ưu chu kỳ máy (CPU cycles) và thời gian cho hệ thống thời gian thực, trường UDP Checksum (offset 14 + IHL + 6) được thiết lập về 0x0000. (Ghi chú: Theo chuẩn RFC 768, UDP Checksum là tùy chọn đối với IPv4).

3.4. Module mã hóa AES

Module mã hóa đóng vai trò cốt lõi trong việc đảm bảo tính bảo mật cho luồng dữ liệu mạng. Vì giới hạn về phần cứng nên hệ thống không thể tích hợp một thư viện mã quá cồng kềnh. Việc sử dụng một phiên bản mã hóa được tinh chỉnh đặc biệt (dựa trên thư viện tiny-AES-c), tối ưu hoàn toàn cho cấu trúc bộ nhớ của vi điều khiển STM32 là hoàn toàn hợp lý.

3.4.1. Đặc tả thuật toán và cấu trúc dữ liệu

Thuật toán lõi là AES (Advanced Encryption Standard) với độ dài khóa 128-bit.

Kích thước khối (Block Size) cố định là 16 bytes (AES_BLOCKLEN = 16).

Hoạt động tại chế độ CBC (Cipher Block Chaining). Trong chế độ này, mỗi khối văn bản thô (Plaintext) được XOR với khối văn bản mã (Ciphertext) trước đó khi đưa vào mã hóa Cipher().

Cấu trúc AES_ctx được sử dụng để duy trì khóa vòng (Round Keys) và Vector khởi tạo (IV).

```
7
8  #define AES_BLOCKLEN 16 // Block length in bytes - AES is 128b block only
9  #define AES_KEYLEN 16   // Key length in bytes
10 #define AES_keyExpSize 176
11
12 typedef struct
13 {
14   uint8_t RoundKey[AES_keyExpSize];
15   uint8_t Iv[AES_BLOCKLEN];
16 } AES_ctx;
17
```

Hình 3.4.1.1: Cấu trúc dữ liệu AES_ctx

3.4.2. Tối ưu hóa bộ nhớ phần cứng

Để phù hợp với giới hạn 20KB RAM của STM32F103C8T6, mã nguồn của module bảo mật sử dụng các kỹ thuật sau:

- Lưu trữ tĩnh S-Box và Rcon: Bảng dò (Lookup Table) sbox[256] và mảng hằng số vòng Rcon[11] được khai báo với từ khóa static const. Kỹ thuật này ép trình biên dịch đặt các mảng dữ liệu này vào bộ nhớ Flash (ROM) thay vì RAM, giúp tiết kiệm đáng kể không gian RAM lúc thực thi.
- Tối ưu hóa phép nhân Galois Field: Macro MULTIPLY_AS_A_FUNCTION được thiết kế để giảm kích thước mã nhị phân (code size) khi biên dịch bằng ARM GCC, giúp quá trình thao tác bit xtime() hiệu quả hơn trong khối MixColumns().

3.4.3. Đặc tả các hàm API chính

a. Hàm khởi tạo ngữ cảnh mã hóa

Tên hàm: AES_init_ctx_iv

Nguyên mẫu: void AES_init_ctx_iv(AES_ctx* ctx, const uint8_t* key, const uint8_t* iv);

Logic thiết kế:

- Tiếp nhận khóa 16-byte (tương ứng với địa chỉ IP nguồn – tra trong bảng định tuyến Key_Table) và IV (Initialization Vector).
- Gọi hàm KeyExpansion để tạo ra lịch trình khóa (Key Schedule), mở rộng khóa 16-byte ban đầu thành mảng Roundkey 176-byte để phục vụ cho 10 vòng mã hóa (Nr = 10) của AES-128.
- Lưu trữ iv vào biến trạng thái ctx→Iv.

b. Hàm mã hóa Payload

Hàm này là trung tâm điều phối, tích hợp trực tiếp cơ chế đệm PKCS#7 vào vùng nhớ của PacketBuffer mà không cần cấp phát thêm bộ đệm phụ.

Tên hàm: AES_CBC_PKCS7_Encrypt

Nguyên mẫu: void AES_CBC_PKCS7_Encrypt(AES_ctx* ctx, PacketBuffer* packet, uint16_t start_offset);

Logic thiết kế:

- Kiểm tra start_offset > packet→length. Nếu offset vượt quá độ dài (gói tin lỗi) thì lập tức hủy bỏ.
- Tính toán kích thước Payload thực tế (payload_length = packet→length – start_offset). Tính số byte cần đệm (padding_length = AES_BLOCKLEN – (payload_length % AES_BLOCKLEN)). (Lưu ý: Ngay cả khi Payload chia hết cho 16, thuật toán vẫn sẽ thêm một khối 16 bytes đệm với giá trị 0x10 để tuân thủ chuẩn giải mã. Để giải quyết vấn đề này kích thước của BUFFER_SIZE sẽ là 614 bytes (lớn hơn 16 bytes so với kích thước gói tin lớn nhất là 598 bytes)).
- Kiểm tra packet→length + padding_length > BUFFER_SIZE. Nếu quá trình đệm dẫn đến việc gói tin vượt quá kích thước BUFFER_SIZE (614 bytes) thì hàm sẽ return ngay lập tức.
- Sử dụng vòng lặp for nối các giá trị đệm vào cuối dữ liệu Payload và tiến hành cập nhật packet→length. Gọi AES_CBC_encrypt_buffer thực hiện mã hóa ngay trên Buffer từ vị trí &packet→data[start_offset]. Việc sử dụng tham chiếu mảng trực tiếp giúp thực thi cơ chế Zero-copy, tiết kiệm chu kỳ CPU.

3.4.4. Trình tự thực thi mã hóa

Quy trình mã hóa một gói tin tuân thủ cấu trúc mạng của lớp 4 (Transport layer):

Quá trình xử lý Cipher() bao gồm 4 phép biến đổi cho mỗi vòng (Round): SubBytes, ShiftRows, MixColumns, AddRoundKey.

Do sử dụng ở chế độ CBC nên quá trình mã hóa là tuyến tính và không thể tính toán song song cho các Block trong cùng một gói tin, do đó đây là Task tiêu tốn tài nguyên CPU nhất trong chu trình hoạt động của vPacket_Processing_Task.

3.5. Thiết kế đa nhiệm

Hệ thống sử dụng hệ điều hành thời gian thực FreeRTOS (thông qua lớp CMSIS-RTOS) để điều phối các luồng xử lý mạng và bảo mật. Các Task được phân bổ mức ưu tiên (Priority) dựa trên mức độ khẩn cấp của phần cứng và ngăn xếp (Stack Size) tĩnh dựa trên khối lượng tính toán.

3.5.1. Danh sách các Task

Tên Task	Mức ưu tiên	Stack Size (Words)	Chức năng
vRX_SPI1_Task	*osPriorityHigh*	256	Phản ứng tức thời từ ngắt phần cứng (EXTI0). Đọc gói tin từ ENC28J60 (Cổng 1), đưa vào xRX_Queue.
vRX_SPI2_Task	*osPriorityHigh*	256	Phản ứng tức thời từ ngắt phần cứng (EXTI1). Đọc gói tin từ ENC28J60 (Cổng 2), đưa vào xRX_Queue.
vTX_Task	*osPriorityAbove Normal*	256	Lấy dữ liệu đã được mã hóa trong xTX_Queue và đẩy ra cáp mạng và thu hồi bộ đệm gói tin.
vPacket_Processing _Task	*osPriorityNormal*	512	Phân tích header Ethernet/IP/UDP, thực hiện việc lọc các gói tin không đạt yêu cầu. Thực hiện mã hóa AES và đẩy gói tin vào Queue xTX_Queue.
vHeartbeat_Task	*osPriorityLow*	128	Giám sát trạng thái của các Task khác, nháy LED và nạp lại Watchdog Timer (IWDG).

vDefault_Task	*osPriorityLow*	128	Task mặc định của hệ thống (Idle Task), xử lý các Task nền.

Bảng 3.5.1.1: Danh sách các Task

3.5.2. Giao diện các Task

Toàn bộ các Task được khởi tạo theo chuẩn CMSIS-RTOS API v1 (osThreadDef). Dưới đây là các nguyên mẫu hàm (prototypes) thực thi vòng lặp vô hạn của từng Task:

```
120 void StartDefaultTask(void const * argument);
121 void vRX_SPI1_TaskFunc(void const * argument);
122 void vRX_SPI2_TaskFunc(void const * argument);
123 void vTX_TaskFunc(void const * argument);
124 void vPacket_Processing_TaskFunc(void const * argument);
125 void vHeartbeat_TaskFunc(void const * argument);
126
```

Hình 3.5.2.1: Nguyên mẫu hàm của các Task

3.5.3. Chi tiết tương tác giữa các Task

Để hệ thống hoạt động ổn định và an toàn, các Task trên tương tác với nhau thông qua cơ chế sau:

a. Cơ chế truyền dữ liệu

vRX_SPIx_Task sau khi nhận được tín hiệu từ xSem_INT_SPIx (có gói tin) thì sẽ gọi đến hàm Rx_HandlePacket. Hàm này sẽ tìm Buffer còn trống cho gói tin, nếu không tìm thấy thì gói tin sẽ bị xóa ngay lập tức, nếu tìm thấy Buffer thì tiến hành nhận gói tin và cập nhật hai thông tin length và source_spi. Sau đó con trỏ trỏ đến gói tin sẽ được đẩy vào xRX_QueueHandle (Timeout 0ms).

vPacket_Processing_Task chờ tại xRX_QueueHandle (Timeout 50ms), xử lý xong thì đẩy tiếp con trỏ qua xTX_QueueHandle (Timeout, 10ms).

vTX_Task lấy con trỏ từ xTX_QueueHandle, gửi xong gọi BufferPool_Release() để tạo Buffer trống cho những gói tin mới.

b. Cơ chế chống treo luồng

Các Task RX, TX và Processing liên tục bật các cờ trạng thái (TASK_ALIVE_RX1, TASK_ALIVE_RX2, TASK_ALIVE_TX, TASK_ALIVE_AES) bên trong vùng Critical Section (taskENTER_CRITICAL()).

vHeartbeat_Task đóng vai trò kiểm tra. Chỉ khi bắt được toàn bộ các cờ này (nghĩa là không có Task nào bị treo/Deadlock), nó mới thực hiện xóa cờ và gọi HAL_IWDG_Refresh(&hiwdg) để ngăn STM32 bị reset cứng.

3.6. Thiết kế Queue

Hệ thống sử dụng cơ chế Queue thông điệp (Message Queue) của FreeRTOS (thông qua lớp CMSIS-RTOS API) để truyền thông liên tiến trình (IPC). Nhiệm vụ của Queue trong kiến trúc này là chuyển giao quyền sở hữu các gói tin giữa các Task một cách an toàn mà không cần sao chép dữ liệu vật lý.

3.6.1. Khởi tạo và cấu trúc bộ nhớ

Để triệt tiêu rủi ro tràn bộ nhớ động, toàn bộ Queue vùng nhớ đệm được cấp phát tĩnh tại thời điểm biên dịch.

```
/* Create the queue(s) */
/* definition and creation of xRX_Queue */
osMessageQStaticDef(xRX_Queue, 6, uint32_t, xRX_QueueBuffer, &xRX_QueueControlBlock);
xRX_QueueHandle = osMessageCreate(osMessageQ(xRX_Queue), NULL);

/* definition and creation of xTX_Queue */
osMessageQStaticDef(xTX_Queue, 4, uint32_t, xTX_QueueBuffer, &xTX_QueueControlBlock);
xTX_QueueHandle = osMessageCreate(osMessageQ(xTX_Queue), NULL);

/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
/* USER CODE END RTOS_QUEUES */
```

Hình 3.6.1.1: Danh sách các Queue và khởi tạo

Lưu ý: Kích thước phần tử là sizeof(uint32_t) (4 bytes) vì Queue chỉ lưu trữ địa chỉ con trỏ của PacketBuffer, không lưu trữ toàn bộ gói tin.

3.6.2. Đặc tả Queue nhận

Là vùng đệm trung chuyển gói tin thô từ giao diện mạng vào Processing Task.

Kích thước của Queue có 6 phần tử. Mức này được phân chia dựa trên lượng PacketBuffer (10 cái) và hệ thống có 2 Task nhận dữ liệu xRX_SPI1_Task và vRX_SPI2_Task.

Luồng dữ liệu:

- vRX_SPI1_Task và vRX_SPI2_Task gọi osMessagePut với Timeout = 0ms. Nếu Queue đầy (hệ thống quá tải), gói tin mới sẽ bị hủy ngay tại phần cứng để tự bảo vệ.
- vPacket_Processing_Task gọi osMessageGet với Timeout = 50ms. Nếu có thông điệp, Task này sẽ được đánh thức ngay lập tức từ trạng thái Blocked.

3.6.3. Đặc tả Queue gửi

Chứa các gói tin đã hoàn tất xử lý (mã hóa, cập nhật header) hoặc các gói tin được phép định tuyến trực tiếp mà không cần mã hóa (ARP, ICMP) chờ đẩy ra mạng.

Kích thước của Queue có 4 phần tử.

Luồng dữ liệu:

- vPacket_Processing_Task sau khi mã hóa xong gói tin sẽ đẩy con trỏ vào Queue với Timeout = 10ms. Nếu Queue gửi bị nghẽn (đầy) lâu hơn 10ms, hệ thống tiến hành gọi BufferPool_Release để hủy gói nhằm giải phóng RAM.
- vTX_Task gọi osMessageGet với Timeout 50ms. Trích xuất thuộc tính source_spi để điều hướng gọi hàm truyền DMA phần cứng.

3.6.4. Các nguyên tắc thiết kế cốt lõi

a. Kiến trúc Zero-copy

Thay vì dùng memcpy để sao chép hàng trăm byte (tối đa 614 bytes) của một gói tin từ Task này sang Task khác, hệ thống chỉ đẩy một con trỏ 32-bit (4 bytes) đi vào Queue. Điều này giúp tiết kiệm chu kỳ CPU vào giảm thời gian trễ khi xử lý gói tin.

b. Kiểm soát nghẽn mạng

Kích thước tổng qua hai Queue là (6 + 4 = 10) được thiết kế cân bằng chính xác với tổng số lượng Buffer trong Static Buffer Pool (10 buffers). Thiết kế này đảm bảo hệ thống không bao giờ xảy ra tình trạng Queue còn chỗ nhưng Pool hết chỗ. Ngoài ra kích thước của Queue nhận (xRX_Queue) lớn hơn kích thước của Queue gửi (xTX_Queue) vì 2 Task nhận có mức ưu tiên cao hơn và có số lượng nhiều hơn Task gửi.

4. THIẾT KẾ ĐA NHIỆM VÀ GIẢI QUYẾT TƯƠNG TRANH

Chương này đặc tả cơ chế thiết kế FreeRTOS và kiểm soát tương tranh nhằm đảm bảo tính toàn vẹn dữ liệu khi nhiều Task và trình phục vụ ngắt (ISR) cùng truy cập vào các tài nguyên phần cứng giới hạn (SPI Bus) và phần mềm (Static Buffer Pool).

4.1. Cơ chế đồng bộ hóa

Hệ thống không sử dụng cơ chế Polling để tiết kiệm năng lượng và tối ưu hóa CPU. Thay vào đó, sự kiện phần cứng được đồng bộ với Task phần mềm thông qua Binary Semaphore.

4.1.1. Đồng bộ ngắt ngoài

Sử dụng xSem_INT_SPI1 và xSem_INT_SPI2.

Khi module ENC28J60 nhận được gói tin mạng, chân INT bị kéo xuống mức thấp, kích hoạt ngắt EXTI. Trình phục vụ ngắt HAL_GPIO_EXTI_Callback lập tức gọi osSemaphoreRelease() để đánh thức vRX_SPIx_Task tương ứng. Task RX đang ở trạng thái ngủ (Blocked) sẽ ngay lập tức chuyển sang trạng thái sẵn sàng để đọc gói tin.

4.1.2. Đồng bộ truyền tải bộ nhớ

Sử dụng xSem_DMA_SPI1_Done và xSem_DMA_SPI2_Done.

Quá trình giao tiếp SPI với ENC28J60 sử dụng DMA để không chặn CPU. Các Task RX/TX sau khi khởi chạy DMA sẽ gọi osSemaphoreWait() để nhường CPU cho các Task khác. Khi DMA hoàn tất, ngắt DMA sẽ giải phóng Semaphore này để Task tiếp tục công việc.

4.2. Cơ chế bảo vệ tài nguyên

Để ngăn chặn tình trạng tương tranh (Race Condition) làm sai lệch dữ liệu, hệ thống sử dụng Mutexes để bảo vệ việc thao tác với dữ liệu trong các khu vực găng (Critical Sections).

4.2.1. Bảo vệ Bus giao tiếp

Sử dụng spi1_mutex và spi2_mutex

Có 4 Task có thể truy cập đến phần cứng mạng cùng lúc: vRX_SPI1_Task và vRX_SPI2_Task (nhận gói tin), vTX_Task (gửi gói tin), vHeartbeat_Task (kiểm tra trạng thái kết nối của vi điều khiển với module ENC28J60). Việc các lệnh SPI đan xen nhau có thể sai lệch dữ liệu và làm hỏng con trỏ nội bộ của ENC28J60 (do nhiều Task cùng truy cập và sửa đổi).

4.2.2. Bảo vệ bộ đệm tĩnh

Sử dụng pool_mutex.

Đảm bảo an toàn cho các hàm BufferPool_Acquire() và BufferPool_Release() kể cả khi vRX_SPI1_Task, vRX_SPI2_Task, vPacket_Processing_Task và vTX_Task gọi chúng đồng thời.

4.3. Cơ chế chống bế tắc

Hệ thống áp dụng nguyên tắc không chờ đợi vô hạn (No Infinite Blocking) cho các tài nguyên có nguy cơ gây treo hệ thống.

4.3.1. Timeout cho Mutex/Semaphore

Các lời gọi osMutexWait cho spi1_mutex và spi2_mutex đều có thời gian chờ từ 50ms – 200ms (50ms cho các công việc như đọc thanh ghi/thanh ghi PHY, 200ms cho việc truyền/nhận gói tin).

Tại hàm truyền/nhận gói tin sử dụng DMA (ENC28J60_SendPacket và ENC28J60_ReceivePacket), nếu xSem_DMA_SPIx_Done không được trả về sau 50ms (do nhiễu điện từ làm mất ngắt DMA), hệ thống lập tức gọi HAL_SPI_DMAStop() để cưỡng chế ngắt DMA, nhả Mutex và kết thúc hàm. Điều này ngăn toàn bộ Bus SPI bị khóa vĩnh viễn.

4.3.2. Loại trừ vòng lặp lồng

Kiến trúc được thiết kế để một Task không bao giờ yêu cầu pool_mutex trong khi đang giữ spi1_mutex hoặc spi2_mutex, loại bỏ gần như hoàn toàn rủi ro Deadlock vòng (Circular Wait).

4.4. Cơ chế giám sát đa nhiệm

Sử dụng bộ định thời giám sát độc lập (IWDG) kết hợp với cờ trạng thái phần mềm để chống hiện tượng treo cục bộ (Starvation/Deadlock ở một luồng).

- Biến toàn cục task_alive_flags chứa 4 bits đại diện cho 4 Task cốt lõi: vRX_SPI1_Task, vRX_SPI2_Task, vPacket_Processing_Task và vTX_Task.
- Trong mỗi chu kỳ làm việc, các Task tự động thiết lập bit của mình vào biến task_alive_flags thông qua block an toàn taskENTER_CRITICAL().
- vHeartbeat_Task (chạy mỗi 500ms) sẽ kiểm tra biến cờ. Chỉ khi cả 4 Task đều đã báo cáo (tức là không có Task nào bị kẹt ở các lệnh Wait), lệnh HAL_IWDG_Refresh(&hiwdg) mới được thực thi.
- Nếu có một Task bị treo, Watchdog (Timeout 3s) sẽ không được làm mới và tự động Reset phần cứng vi điều khiển, đảm bảo Gateway luôn tự phục hồi sau sự cố phần mềm.

4.4.1. Cơ chế lập lịch

Nhân (Kernel) FreeRTOS trong hệ thống được cấu hình sử dụng thuật toán lập lịch ưu tiên độc chiếm (Preemptive Priority-based Scheduling) kết hợp với chia sẻ thời gian (Time Slicing) để điều phối CPU.

4.4.2. Lập lịch ưu tiên độc chiếm

Hệ thống tuân thủ nguyên tắc: CPU luôn được cấp cho Task có mức ưu tiên cao nhất (Highest Pritority) đang ở trạng thái sẵn sàng hoạt động (Ready).

Giả sử vPacket_Processing_Task (mức ưu tiên osPriorityNormal) đang chiếm CPU để thực hiện thuật toán AES mã hóa Payload. Khi có một gói tin mới được gửi tới ENC28J60, kích hoạt ngắt EXTI. Ngay lập tức, vRX_SPIx_Task (mức ưu tiên osPriorityHigh) chuyển sang trạng thái Ready. Nhân FreeRTOS sẽ tự động dừng vPacket_Processing_Task (có lưu lại trạng thái để quay lại thực hiện tiếp) và cấp CPU cho vRX_SPIx_Task để tiến hành đọc gói tin vào RAM. Sau khi đọc xong, Task RX tự đưa mình vào trạng thái ngủ (Blocked) và CPU được cấp cho vPacket_Processing_Task để thực hiện nốt công việc.

Với cơ chế này đảm bảo độ trễ phản hồi phần cứng là thấp nhất, triệt tiêu rủi ro tràn bộ đệm vật lý của chip mạng.

4.4.3. Lập lịch đồng mức ưu tiên

Trong hệ thống, hai Task vRX_SPI1_Task và vRX_SPI2_Task được thiết kế với cùng một mức ưu tiên cao nhất (osPriorityHigh).

Khi cả hai Task này cùng ở trạng thái Ready (ví dụ: hai cổng mạng đều có gói tin đến cùng một lúc), FreeRTOS sẽ áp dụng thuật toán lập lịch Round-Robin. CPU sẽ luân phiên chia sẻ thời gian xử lý cho hai Task này sau mỗi nhịp đếm của hệ điều hành (Tick Interrupt). Điều này đảm bảo tính công bằng (Fairness), không có cổng mạng nào bị bỏ đói (Starvation) dẫn đến rớt gói tin.

4.4.4. Quản lý trạng thái và tối ưu hóa CPU

Thiết kế của hệ thống tuân thủ nguyên tắc hướng sự kiện (Event-driven) và không sử dụng cơ chế Polling để đợi sự kiện. Điều này được thể hiện qua trạng thái hoạt động của các Task.

- Trạng thái ngủ (Blocked State): Hầu hết thời gian của các Task cốt lõi đều nằm ở trạng thái này. Chúng ngủ và không tiêu tốn bất kỳ chu kỳ CPU nào khi chờ sự kiện (osSemaphoreWait hoặc osMessageGet).
- Trạng thái sẵn sàng (Ready State): Chỉ chuyển sang trạng thái này khi có sự kiện xảy ra (có ngắt vật lý hoặc có địa chỉ của gói tin được đẩy vào Queue).
- Trạng thái thực thi (Running State): Chỉ có duy nhất 1 Task được chạy tại một thời điểm.
- Task nhàn rỗi (Idle Task): Khi không có gói tin nào qua mạng (tất cả các Task đều Blocked), hệ thống sẽ tự chạy defaultTask với mức ưu tiên thấp nhất (osPriorityLow). Tại đây, CPU chỉ chạy hàm osDelay(1) (delay 1ms cho mỗi phiên làm việc).

5. BIỂU ĐỒ TUẦN TỰ

Biểu đồ tuần tự dưới đây mô tả vòng đời (lifecycle) của một gói tin từ khi được nhận tại giao diện mạng vật lý cho đến khi nó được xử lý mã hóa và chuyển tiếp đi, thể hiện rõ cơ chế giao tiếp Zero-copy qua các Queue.

5.1. Mô tả tương tác hệ thống

Một gói tin khi đến hệ thống sẽ được xử lý theo 9 bước dưới đây.

Bước 1: Module ENC28J60 nhận gói tin và kéo chân ngắt (EXTI) xuống mức thấp để đánh thức vRX_SPIx_Task.

Bước 2: vRX_SPIx_Task gọi hàm BufferPool_Acquire() để lấy một vùng nhớ tĩnh cho gói tin.

Bước 3: vRX_SPIx_Task sử dụng hàm ENC28J60_ReceivePacket để đọc gói tin từ chip mạng vào bộ đệm được cấp phát qua Bus SPI.

Bước 4: vRX_SPIx_Task đẩy con trỏ của Buffer vào xRX_Queue để nhường quyền xử lý cho vPacket_Processing_Task.

Bước 5: vPacket_Processing_Task lấy con trỏ ra khỏi Queue.

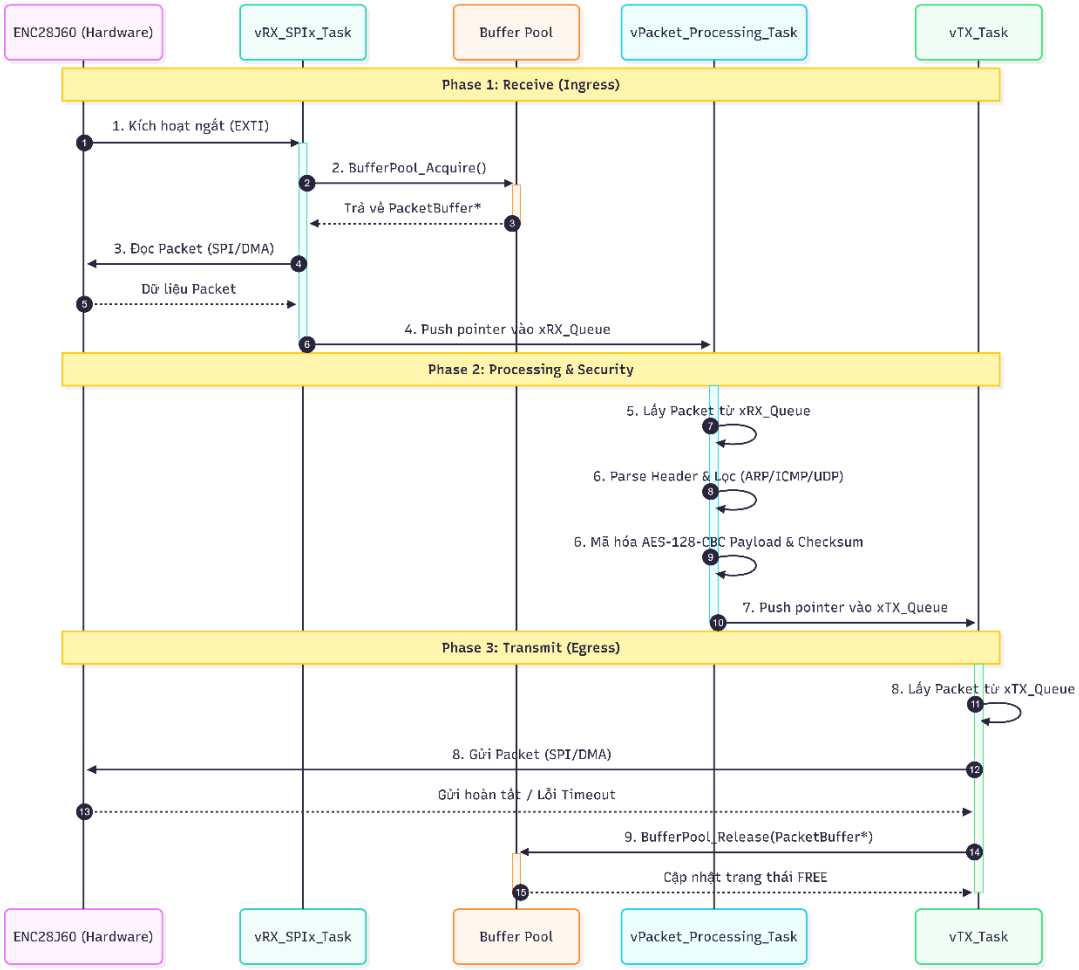
Bước 6: vPacket_Processing_Task phân tích (Parse) Header và mã hóa (Encrypt) Payload bằng AES-128-CBC trực tiếp ngay trên Buffer.

Bước 7: vPacket_Processing_Task đẩy con trỏ vào xTX_Queue.

Bước 8: vTX_Task lấy con trỏ, ra lệnh cho DMA/SPI đẩy dữ liệu từ bộ đệm ra module ENC28J60.

Bước 9: Sau khi truyền xong, vTX_Task gọi BufferPool_Release() để trả bộ đệm về trạng thái rảnh.

5.2. Biểu đồ UML



Hình 5.2.1: Biểu đồ UML quá trình xử lý một gói tin

6. THIẾT KẾ VÀ PHÂN BỔ BỘ NHỚ

Hệ thống được thiết kế theo nguyên tắc quản lý bộ nhớ tĩnh, loại bỏ hoàn toàn cấp phát động để đảm bảo tính ổn định thời gian thực và tránh hiện tượng phân mảnh RAM.

6.1. Phân bổ bộ nhớ RAM

Tổng dung lượng RAM của vi điều khiển là 20 KB. Hệ thống sử dụng khoảng 17.35 KB (tương đương ~87% tài nguyên RAM), phần dư thừa được giữ lại để dự phòng cho các hoạt động ngắt (Interrupt) và tránh lỗi Stack Overflow.

Thành phần	Chi tiết	Dung lượng thực tế
Static Buffer Pool	10 Buffers x 614 bytes	6.0KB
Task Stacks	Cấp phát tĩnh cho 7 Tasks (bao gồm Default và Timer Task)	6.38KB
FreeRTOS Heap & Queue	Quản lý Queue (xRX_Queue, xTX_Queue) và Semaphores	1.1KB
HAL & Global Variables	Các biến handle ngoại vi, bảng định tuyến, trạng thái hệ thống...	3.87KB
Tổng cộng chiếm dụng		17.35KB

Bảng 6.1.1: Phân bổ dung lượng RAM

6.2. Phân bổ bộ nhớ chương trình

Với 64 KB bộ nhớ Flash có sẵn, hệ thống chiếm dụng khoảng 33.75 KB (~53%). Cho phép khả năng mở rộng thêm các tính năng bảo mật trong tương lai.

Thành phần	Chi tiết	Dung lượng thực tế

FreeRTOS Kernel	Nhân điều phối hệ điều hành và thư viện CMSIS-RTOS.	8.56KB
STM32 HAL Drivers	Thư viện trừu tượng hóa phần cứng cho SPI, DMA, IWDG.	9.44KB
Tiny-AES & Logic	Thư viện Tiny-AES, logic phân tích gói tin và tính Checksum	5.24KB
ENC28J60 Driver	Trình điều khiển module mạng và xử lý Silicon Errata	7.07KB
Startup & Libc	Mã khởi tạo hệ thống và các hàm thư viện C cơ bản	3.44KB
Tổng cộng chiếm dụng		33.75KB

Bảng 6.2.1: Phân bổ dung lượng Flash

7. CƠ CHẾ XỬ LÝ LỖI VÀ NGOẠI LỆ

7.1. Các điều kiện hủy gói tin

Hệ thống sẽ chủ động từ chối phục vụ và hủy gói tin khi gói tin vi phạm một trong ba nhóm điều kiện sau.

7.1.1. Cạn kiệt bộ đệm tĩnh

Nguyên nhân do lưu lượng mạng đến quá nhanh (vượt quá khả năng xử lý của CPU), dẫn đến toàn bộ 10 phần tử trong pool_status đều ở trạng thái BUFFER_IN_USE.

Lỗi được phát hiện bởi hàm BufferPool_Acquire() khi nó trả về NULL.

Nếu không xử lý ngoại lệ này thì vùng Buffer đang được xử lý bởi một Task khác có rủi ro bị ghi đè bởi dữ liệu gói tin mới.

7.1.2. Quá tải Queue

Nguyên nhân do tài nguyên hạn chế của STM32F103C8T6 (chỉ 20KB RAM) khiến cho xRX_Queue chứa được tối đa 6 con trỏ và xTX_Queue chứa được tối đa 4 con trỏ.

Lỗi này sẽ được phát hiện khi hệ thống đẩy con trỏ của gói tin vào Queue (sử dụng osMessagePut()) trả về mã lỗi (!= osOK) do Timeout (10ms) hết hạn và vẫn chưa còn chỗ trống.

Nếu không xử lý ngoại lệ này (không thiết lập Timeout) thì Task đẩy dữ liệu vào sẽ bị block vĩnh viễn và gây treo hệ thống.

7.1.3. Gói tin không hợp lệ

Nguyên nhân do gói tin bị lỗi định dạng vật lý, sai giao thức.

Phát hiện những lỗi này trong quá trình phân tích Header tại vPacket_Processing_Task.

- Phần Payload của gói tin vượt quá kích thước cho phép (> 512 bytes).
- Sai chuẩn Ethernet (EtherType != 0x0800 hoặc 0x0806).
- Không phải giao thức UDP/ICMP hoặc gói UDP bị phân mảnh (Fragmented).
- Địa chỉ IP nguồn (Source IP) không có trong bảng định tuyến tĩnh (key_table).

7.2. Chiến lược phục hồi

Chiến lược phục hồi của Gateway là Zero-Retry (Không thử gửi lại). Do đặc điểm của giao thức UDP là không đảm bảo độ tin cậy (Unreliable), việc loại bỏ gói tin trên Gateway là hoàn toàn hợp lệ theo chuẩn mạng.

7.2.1. Phục hồi ngay lập tức

Khi bất kỳ điều kiện lỗi nào ở mục 7.1 xảy ra, quy trình phục hồi tài nguyên được kích hoạt ngay lập tức mà không cần chờ CPU xử lý thêm.

Trường hợp lỗi tại tầng mạng vật lý (hết Buffer) thì vRX_SPIx_Task gọi ENC28J60_DropPacket(). Hàm này can thiệp trực tiếp vào thanh ghi của chip mạng để dời con trỏ đọc (Read Pointer) bỏ qua gói tin hiện tại, ép chip mạng hủy gói tin ngay trên SRAM phần cứng, bảo vệ an toàn cho RAM của STM32.

Trường hợp lỗi tại tầng phần mềm (Queue đầy hoặc gói tin không hợp lệ) thì Task đang xử lý gói tin sẽ gọi hàm BufferPool_Release(). Việc này đảo trạng thái bộ đệm về BUFFER_FREE, nhường chỗ cho các gói tin hợp lệ tiếp theo đi vào hệ thống, chặn đứng nguy cơ rò rỉ bộ nhớ (Memory Leak).

7.2.2. Cơ chế không thử lại

Hệ thống tuyệt đối không có vòng lặp thử lại để cố gắng đẩy lại con trỏ vào Queue hoặc xin lại Buffer.

Việc chờ đợi sẽ làm tăng thời gian block của các Task có mức ưu tiên cao như (vRX_SPIx_Task), làm mất tính thời gian thực của hệ thống và tăng khả năng bị nghẽn CPU.

8. MA TRẬN TRUY VẾT YÊU CẦU

Chương này cung cấp ánh xạ hai chiều giữa Tài liệu đặc tả yêu cầu phần mềm (SRS) và Tài liệu mô tả thiết kế phần mềm (SDD). Mục đích của ma trận là đảm bảo tính toàn vẹn của hệ thống, hỗ trợ quá trình kiểm thử và đánh giá mức độ đạt yêu cầu của thiết kế.

Mã FR (SRS)	Mô tả yêu cầu (Tóm tắt)	Module thiết kế (SDD)	Task/Hàm thực thi

FR-01	Nhận gói tin Ethernet thô qua SPI.	Module Driver ENC28J60.	vRX_SPIx_TaskFunc() ENC28J60_ReceivePacket()
FR-02	Phân tích cấu trúc gói tin (Header).	Module phân tích và định tuyến gói tin.	vPacket_Processing_TaskFunc() (Logic bóc tách Lớp 2/3/4)
FR-03	Kiểm tra tính hợp lệ và lọc gói tin.	Module phân tích và định tuyến gói tin.	vPacket_Processing_TaskFunc() (Kiểm tra Fragment và giao thức)
FR-04	Mã hóa Payload bằng AES-128-CBC.	Module mã hóa AES.	AES_CBC_PKCS7_Encrypt() Bảng định tuyến key_table
FR-05	Tính toán lại Checksum và đóng gói.	Module phân tích và định tuyến gói tin.	Update_Ip_Checksum()
FR-06	Chuyển tiếp cổng.	Module phân tích và định tuyến gói tin. Thiết kế Queue. Module ENC28J60 Driver.	vPacket_Processing_TaskFunc() Queue xTX_Queue vTX_TaskFunc ENC28J60_SendPacket()
FR-07	Quản lý bộ đệm, tránh phân mảnh RAM.	Module quản lý bộ đệm.	BufferPool_Acquire() BufferPool_Release()

Bảng 8.1: Ma trận truy vết các yêu cầu chức năng (FR)

Mã NFR (SRS)	Tiêu chí yêu cầu	Giải pháp thiết kế	Chương tham chiếu
NFR-01	$RAM \leq 90\%$	Sử dụng Static Buffer Pool và cơ chế Zero-copy.	Chương 3.2 và 6.1
NFR-02	$Flash \leq 64\ KB$	Lưu trữ S-Box và hằng số vòng AES vào ROM (Flash).	Chương 4.3.1 và 6.2
NFR-03	Độ $trễ < 5ms$	Sử dụng DMA cho SPI và lập lịch ưu tiên độc chiếm (Preemptive).	Chương 3.1 và 4.5
NFR-04	$Load \geq 50\ gói/s$	Cơ chế hàng đợi và đồng bộ hóa bằng Semaphore.	Chương 3.6 và 4.1
NFR-05	$Throughput \approx 239\ Kbps$	Tối ưu hóa chu kỳ CPU cho AES và tính toán Checksum.	Chương 3.4 và 3.3

Bảng 8.2: Ma trận truy vết các yêu cầu phi chức năng (NFR)

Mã ADR (SAD)	Quyết định kiến trúc	Module/Hàm triển khai	Mô tả cách thực thi
ADR-01	Sử dụng FreeRTOS	Chương 3.5 và 4.5	Triển khai 6 Task với các mức ưu tiên khác nhau để điều phối luồng dữ liệu. Sử dụng osThreadStaticDef để cấp phát stack tĩnh.

ADR-02	Cấp phát bộ nhớ tĩnh	Chương 3.2 và 6.1	Khai báo mảng pool_buffers[10] và pool_status[10]. Không sử dụng malloc() và free() trong mã nguồn.
ADR-03	Kỹ thuật Zero-copy	Chương 3.6 và 4.3	Chỉ truyền địa chỉ con trỏ PacketBuffer* qua xRX_Queue và xTX_Queue thay vì sao chép toàn bộ mảng dữ liệu.

Bảng 8.3: Ma trận truy vết quyết định kiến trúc (ADR)

9. HẠN CHẾ CỦA HỆ THỐNG VÀ ĐÁNH ĐỔI THIẾT KẾ

Do rào cản về tài nguyên phần cứng (20KB RAM, 64KB Flash, CPU 72MHz) và yêu cầu đảm bảo độ trễ thời gian thực thấp, kiến trúc của hệ thống Secure Gateway hiện tại chấp nhận một số đánh đổi về mặt tính năng và bảo mật.

9.1. Giới hạn giao thức hỗ trợ

Hệ thống hiện tại chỉ bóc tách, định tuyến và mã hóa các luồng dữ liệu phi trạng thái như UDP và ICMP. Giao thức TCP bị loại bỏ hoàn toàn khỏi luồng xử lý.

Việc hỗ trợ gói tin TCP đòi hỏi hệ thống phải theo dõi trạng thái kết nối (TCP State Machine), quản lý cửa sổ trượt (Sliding Windows), xác nhận gói tin (ACK) và đặc biệt là duy trì một vùng đệm truyền lại khổng lồ (Retransmission Buffer) cho các gói tin bị lỗi trong quá trình truyền. Điều này sẽ nhanh chóng làm cạn kiệt 20KB RAM của STM32F103C8T6.

9.2. Rủi ro từ Vector khởi tạo cố định

Thuật toán AES-128-CBC yêu cầu một Vector khởi tạo (IV) ngẫu nhiên và duy nhất cho mỗi phiên mã hóa. Tuy nhiên, thiết kế hiện tại chỉ sử dụng một IV cố định (iv[16] = {0}) cho tất cả các gói tin.

Điều này làm suy yếu đặc tính phân tán của chế độ CBC. Nếu hai gói tin mạng có phần bản rõ (Plaintext) giống hệt nhau, chúng sẽ tạo ra phần bản mã (Ciphertext) cũng giống hệt nhau. Kẻ tấn công có thể dựa vào đặc điểm này để phân tích dữ liệu mạng.

Để xử lý IV ngẫu nhiên, hệ thống bắt buộc phải đính kèm đoạn IV 16-byte đó vào mỗi gói tin gửi đi để thiết bị đích có thể giải mã. Việc này làm tăng kích thước của gói tin và đòi hỏi phải tự định nghĩa lại cấu trúc của Payload UDP, gây phức tạp cho quá trình triển khai.